

Fast Proximal Optimization for Sparse Reconstruction with Dictionaries Based on Translated Waveforms

Tom Trigano¹, Shira Vaknin¹, David Luengo^{2,*}

Abstract

The use of compressive sensing tools has become frequent in signal processing applications when dealing with sparse signals. It has been recently shown that even in the case of very correlated dictionaries, such as dictionaries based on translated waveforms, information can be retrieved from the observations with accuracy. In a recent paper, the authors presented Convolutional Sparse Approximation (CoSA), a fast version of the *Iterative Shrinkage and Thresholding Algorithm* (ISTA), which is well suited for dictionaries based on this type of waveforms. In this paper, we show that CoSA is indeed a specific case of a much more general framework, and we also present three major practical improvements. Firstly, we show that the acceleration scheme can be fastened for proximal gradient methods by automatically computing the gradient's step size in a fast and efficient way, based on the dictionary at hand, thus circumventing the manual, trial and error, setups. Secondly, we show that this method can be used on many more penalized regression problems, and illustrate this aspect on Group LASSO. Eventually, we show that this acceleration procedure can be adapted for the alternating direction method of multipliers (ADMM) optimization scheme, which is often used in the field of compressive sensing for a large class of optimization problems. Results on simulations illustrate the efficiency of the method.

*Corresponding author

¹Department of Electrical Engineering, Shamoon College of Engineering (SCE), Ashdod campus, Israel

²Department of Signal Theory and Communications, Universidad Politécnica de Madrid (UPM), Madrid, Spain

Keywords: Compressive sensing; Sparse reconstruction; Proximal optimization; Fast Iterative Shrinkage and Thresholding Algorithm (FISTA); Fast Fourier Transform (FFT).

1. Introduction

Since the development of compressive sensing theory and applications, the use of convex optimization techniques has become more and more popular. Among other techniques, methods based on thresholded gradients [1] converge to a sparse solution quickly [2, 3] and have been widely used in signal processing applications [4]. Indeed, in many applications (such as spectroscopy [5–7], electrocardiographic (ECG) signal processing [8], network traffic analysis [9] or image processing [10]) we are interested in the sparse reconstruction of a recorded signal. In order to do so, it is commonly accepted to build the sparse reconstruction using incoherent dictionaries, i.e., dictionaries which include very uncorrelated shapes. However, in numerous frameworks, such as nuclear spectroscopy and ECG signal processing, it has been shown empirically (e.g., see [5, 11–16]) that we can rely on correlated dictionaries composed of multiple translated waveforms and still obtain good results. Furthermore, recent developments in the field include theoretical investigations on correlated dictionaries [6, 17]. Unfortunately, the algorithms used are typically cumbersome and rely on extremely large dictionaries due to the sizes of the data sets at hand. In a recent contribution, [18] provided a fast algorithm to deal with large signals using the Iterative Shrinkage-Thresholding Algorithm (ISTA). Based on a digital signal processing trick, it was shown that the most time-consuming operations could be computed quickly and efficiently using zero-padded Fast Fourier Transform (FFT) multiplications. The resulting gain in speed and the ability to deal with large data sets without the need to actually store a large dictionary in memory make the resulting algorithm, Convolutional Sparse Approximation (CoSA), very attractive for real-time or quasi-real time applications. However, CoSA is limited to LASSO, and the gradient’s step size has to be set manually,

27 which could make the procedure either converge very slowly or diverge if poorly
28 set.

29 The main objective of the present paper is to extend [18] from several points
30 of view. More specifically:

- 31 • The algorithm presented in [18] was an accelerated proximal gradient with
32 a user-defined gradient step. We show in this paper that this method
33 can be further improved by choosing a close-to-optimal gradient step by
34 adapting the acceleration scheme for power iterations. It is important to
35 notice that power iterations cannot be performed for very large signals or
36 dictionaries.
- 37 • In [18], we illustrated the acceleration on LASSO [19]. We show here that
38 this acceleration can be also used for all penalized regression problems.
39 We aim to solve these problems using a proximal gradient method, and
40 illustrate this fact on Group LASSO [20].
- 41 • Another well-known and widely used method to solve optimization prob-
42 lems in the field of compressed sensing is the alternating direction method
43 of multipliers (ADMM) [21]. This method is known to converge in less
44 iterations than projected gradient descents. While [18] deals only with
45 projected gradient methods, we present a adaptation of our acceleration
46 technique to ADMM in this paper.

47 In this paper, we show that all proximal gradients based methods can benefit
48 from the proposed acceleration, and that this method can also be implemented
49 for ADMM, in a less general setting, for problems defined later on as *homo-*
50 *geneous*. This class of problems is large enough to be interesting in practice,
51 and encompasses numerous well-known minimization problems, such as LASSO
52 and Group LASSO. Furthermore, the proposed method does not need to save
53 the full dictionary in memory, thus allowing us to process large signals without
54 breaking them into smaller chunks as required by other methods.

55 The rest of the paper is organized as follows: Section 2 presents the class

of penalized regression problems investigated, and recalls the main algorithms used for their solution. We then show how the main bottlenecks of previously described algorithms can be circumvented for dictionaries stemming from translated and known waveforms. Results, obtained on simulations using synthetic signals, illustrate the gain in speed achieved by using the proposed methods without compromising the reconstruction quality.

2. Sparse Reconstruction of Shot-Noise Models Based on Proximal Methods

We describe in this section the model investigated, as well as the fundamentals of the types of algorithms used [3, 21, 22].

2.1. Model Description

The model considered in this paper assumes that we have N points of a sampled signal, y_n , composed of electrical pulses of different shapes and amplitudes [5, 13, 14],

$$y_n = \sum_{k=-\infty}^{\infty} E_k \Phi_k(t_n - T_k) + \varepsilon_n, \quad n = 0, \dots, N-1,$$

where T_k denotes the starting time of the k -th electrical pulse; E_k denotes its amplitude; Φ_k denotes the associated, unknown, pulse shape; and ε_n denotes an additional noise, assumed to be Gaussian and with known variance throughout the paper. Eq. (??) is known as the *shot-noise model*. It is commonly used, among others, in the field of nuclear spectroscopy to model signals stemming from Germanium detectors, as well as in biomedical engineering to model spiky biosignals. In standard measurements, neither Φ_k nor T_k and E_k are known. However, it is common to dispose of prior knowledge on the formation of Φ_k , and therefore to approximate it by means of a dictionary of known waveforms tailored to the problem at hand.

More precisely, we define a set of p candidate waveforms to characterize the signal dynamics as Γ_s ($s = 1 \dots p$), all of them with a finite support of

$M < N$ sampling points along the time scale. Along the paper, we assume that $p < N$, which is a plausible assumption in the framework of sparse reconstruction. The shapes Γ_s are used to estimate the local behavior of the signal around all the sampling points. From [18], it can be noticed that (??) can be rewritten as a regression problem. Namely, we define a *shape-block* matrix $\mathbf{A}_s = [\Gamma_s(t_i - t_k)]_{0 \leq i \leq N-1, 0 \leq k \leq N-M-1}$, and a shape based $N \times (N - M)p$ dictionary as $\mathbf{A} = [\mathbf{A}_1 \ \mathbf{A}_2 \ \cdots \ \mathbf{A}_p]$. Neglecting the influence of the approximation made between the dictionary's shapes and the actual shapes Φ_k (a discussion on this influence is provided in [6]), Eq. (??) can therefore be rewritten as

$$\mathbf{y} = \mathbf{A}\boldsymbol{\beta} + \boldsymbol{\varepsilon} = \sum_{s=1}^p \mathbf{A}_s \boldsymbol{\beta}_s + \boldsymbol{\varepsilon},$$

where $\mathbf{y} = [y_0, y_1, \dots, y_{N-1}]^T$ and $\boldsymbol{\varepsilon} = [\varepsilon_0, \varepsilon_1, \dots, \varepsilon_{N-1}]^T$ represent, respectively, the recorded signal and the additive noise, and $\boldsymbol{\beta}$ is a sparse vector whose non-null entries are related to occurrences of the events of interest (photon arrival times in spectroscopy and heartbeat times in electrocardiography).

In numerous signal processing applications, we are interested in estimating the occurrences of rare events. For example, in the field of nuclear spectroscopy the times T_n form a sample path of a Poisson process, and it is necessary to estimate its activity [5, 6]. A similar problem appears in the study of ECGs, where we are interested in finding the position of the relevant electrical pulses (associated to the depolarization and repolarization of cardiac cells) in each lead of the recording [13, 14, 16]. In all cases, it is important to provide a reconstruction of the signal $\mathbf{y}$ that enforces sparsity. In previous contributions, (??) was investigated using standard sparse reconstruction techniques, such as the Least Absolute Shrinkage and Selection Operator (LASSO) [19] and the group LASSO [20]. In its general form, we aim to solve the following optimization problem:

$$\begin{aligned}
 \hat{\boldsymbol{\beta}} &= \arg \min_{\boldsymbol{\beta}} PR_{\lambda, \mathbf{A}}(\boldsymbol{\beta}) \\
 &= \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2N} \|\mathbf{y} - \mathbf{A}\boldsymbol{\beta}\|_2^2 + \lambda \text{Pen}(\boldsymbol{\beta}) \right\},
 \end{aligned}$$

where λ is a parameter defining the trade off between the sparsity of β and the precision of the estimation, and $\text{Pen}(\beta)$ is a penalty term added to enforce sparsity (typically, the ℓ_1 -norm for LASSO, its grouped version for group LASSO, and a mixture of ℓ_1 and ℓ_2 norms for the Elastic Net). To ensure that the problem is solvable, at least from a theoretical point of view, the only condition imposed on (??) is the convexity of $\text{Pen}(\beta)$. Note that $\text{Pen}(\beta)$ is not necessarily smooth nor differentiable everywhere. In the compressive sensing framework, solving (??) is usually addressed either by means of proximal gradient methods or using an ADMM approach. Both algorithms belong to a large family of optimization techniques which can be used to solve numerous convex optimization problems: *proximal algorithms*.

It might seem restrictive to focus on dictionaries made of translated waveforms. However, the optimization problem described in this paper is strongly related to the the Convolutional Sparse Coding (CSC) model, as presented in [23]. Among other fields, this model has been successfully applied in neuroscience [24], target tracking [25], source separation [26] and music signal processing [27]. In order to apply this model to real-time problems, the acceleration of the optimization by means of linear algebra results has been discussed in [10, 28, 29]. In this paper, we show that an acceleration can also be found for a more general class of problems, and when compared to previous methods, that an additional gain of speed can be obtained up to a mild matrix approximation.

Before describing both algorithms in more detail, we state an essential definition for all the acceleration schemes proposed in this paper.

Definition 1 (Homogeneity). *Given a penalized regression functional to be minimized $PR_{\lambda, \mathbf{A}}$, we shall say that PR is homogeneous if for all $\alpha > 0$, the fact that $\hat{\beta}$ is a minimizer of $PR_{\lambda, \mathbf{A}}$ implies that $\hat{\beta}/\alpha$ is a minimizer of $PR_{\alpha\lambda, \alpha\mathbf{A}}$.*

Simply stated, homogeneity is a property which guarantees the equivalence between the minimization of the functional $PR_{\lambda, \mathbf{A}}$ and the minimization of $PR_{\alpha\lambda, \alpha\mathbf{A}}$, i.e., the invariance of (??) with respect to a scaling constant $\alpha > 0$. Note that this property is useful for the numerical implementation of the

138 optimization methods described later, since it allows some flexibility on the
 139 dictionary $\mathbf{A}$ and its associated Gram matrix $\mathbf{A}^T \mathbf{A}$. In particular, homogeneity
 140 allows control of the spectral radius of $\mathbf{A}^T \mathbf{A}$ while still being able to derive a
 141 minimizer $\hat{\beta}$ of the original problem, since it can be recovered by multiplying
 142 the solution of $PR_{\alpha\lambda, \alpha\mathbf{A}}$ by the user-defined scaling parameter α . This point
 143 is critical for some of the implementation improvements proposed. Moreover,
 144 the property of homogeneity is non-trivial, and does not necessarily hold. As an
 145 example, the ordinary Least Squares, LASSO and Group LASSO functionals are
 146 homogeneous, whereas Tikhonov, ℓ_q -norm for $q < 1$ and Elastic Net functionals
 147 are not homogeneous.

148 2.2. The Proximal Gradient and ADMM for Sparse Reconstruction

149 As previously detailed, we aim to solve the optimization problem

$$\arg \min_{\beta} \left\{ f(\beta) + \lambda \text{Pen}(\beta) \right\},$$

150 where $f(\beta) = \frac{1}{2N} \|\mathbf{A}\beta - \mathbf{y}\|_2^2$ denotes the standard convex objective cost function,
 151 differentiable with Lipschitz continuous gradient ∇f and Lipschitz constant L ;
 152 and Pen the penalty, which is a continuous and convex function, not necessarily
 153 differentiable everywhere. In the sparse reconstruction framework, $\text{Pen}(\beta)$ is
 154 the penalty introduced to enforce sparsity (e.g., $\text{Pen}(\beta) = \|\beta\|_1$ for LASSO,
 155 and $\text{Pen}(\beta) = \sum_{g \in \mathcal{G}} \|\beta_g\|_2$ for grouped LASSO – in the latter, $\mathcal{G}$ represents
 156 the partition of the indexes of the β 's into non-overlapping groups). In the
 157 compressive sensing literature, the minimization (??) is usually solved by means
 158 of one of the two following algorithms: proximal gradient or ADMM. Both rely
 159 on the iterative computation of one or two proximal operators. Their definition
 160 is briefly recalled here.

161 Given any convex, possibly non-smooth, function g , with domain of defini-
 162 tion D_g , to be minimized, we denote the proximal operator by

$$\mathbf{prox}_g(\mathbf{v}) = \arg \min_{\mathbf{x}} \left\{ g(\mathbf{x}) + \frac{1}{2} \|\mathbf{x} - \mathbf{v}\|_2^2 \right\}.$$

163 A proximal gradient algorithm to obtain the solution of (??) is based on
 164 iterated computations of the proximal operator with $g = \lambda \text{Pen}$, that is:

$$\begin{cases} \text{Initialize} & \boldsymbol{\beta}^{(0)} \\ \text{Repeat} & \boldsymbol{\beta}^{(n+1)} = \mathbf{prox}_{\mu\lambda\text{Pen}}(\boldsymbol{\beta}^{(n)} - \mu\nabla f(\boldsymbol{\beta}^{(n)})), \end{cases}$$

165 where μ in (??) denotes a positive non-increasing step size, possibly constant.
 166 As shown in [3], this iterative scheme is guaranteed to converge to $\hat{\boldsymbol{\beta}}$, provided
 167 that ∇f is L -Lipschitz, that is $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|_2 \leq L\|\mathbf{x} - \mathbf{y}\|_2$, for all $\mathbf{x}, \mathbf{y}$,
 168 and provided that $\mu \leq \frac{1}{L}$. For the sparse reconstruction framework, $f(\boldsymbol{\beta}) =$
 169 $\frac{1}{2N}\|\mathbf{A}\boldsymbol{\beta} - \mathbf{y}\|_2^2$ and $\nabla f(\boldsymbol{\beta}) = \frac{1}{N}\mathbf{A}^T(\mathbf{A}\boldsymbol{\beta} - \mathbf{y})$, so we can easily see that L is the
 170 highest eigenvalue of $\frac{1}{N}\mathbf{A}^T\mathbf{A}$. Consequently, in that case the optimal value of
 171 μ is constant and known. However, let us remark that the computation of the
 172 spectral radius of $\mathbf{A}^T\mathbf{A}$ can be very time consuming in practice. Section 3.2
 173 describes a fast and efficient way of estimating it. Moreover, the convergence
 174 of (??) can be accelerated when the same idea is applied to a judiciously chosen
 175 linear combination of $\boldsymbol{\beta}^{(n)}$ and $\boldsymbol{\beta}^{(n-1)}$. This leads to the FISTA approach [3],
 176 whose rate of convergence is better than the direct application of (??) by an
 177 order of magnitude:

Data: $\lambda > 0, 0 < \mu < 1/L$

Result: $\hat{\boldsymbol{\beta}}$, a solution of (??)

Initialization: set $\mathbf{y}^{(0)} = \boldsymbol{\beta}^{(0)}, t_0 = 1$;

while *Convergence is not attained* **do**

Compute;

$\boldsymbol{\beta}^{(n+1)} = \mathbf{prox}_{\mu\lambda\text{Pen}}(\mathbf{y}^{(n)} - \mu\nabla f(\mathbf{y}^{(n)}));$

$t_{n+1} = \frac{1 + \sqrt{1 + 4t_n^2}}{2};$

$\mathbf{y}^{(n+1)} = \boldsymbol{\beta}^{(n+1)} + \left(\frac{t_n - 1}{t_{n+1}}\right)(\boldsymbol{\beta}^{(n+1)} - \boldsymbol{\beta}^{(n)});$

end

Algorithm 1: The FISTA optimization method [3].

178 A second, often used, algorithm is ADMM, which provides a faster way than

179 FISTA in terms of the number of required iterations to solve (??) [21]. However,
 180 this gain is often counterbalanced by preliminary computations (matrix inver-
 181 sion or multiplication) which turn out to be time-consuming even if performed
 182 only once. The ADMM procedure is summarized in algorithm 2.

Data: $\lambda > 0, \mu > 0$

Result: $\hat{\beta}$, a closed-form solution of (??)

while *Convergence is not attained* **do**

 Compute;

$\beta^{(n+1)} = \text{prox}_{\mu f}(\mathbf{z}^{(n)} - \mathbf{u}^{(n)});$

$\mathbf{z}^{(n+1)} = \text{prox}_{\mu \lambda \text{Pen}}(\beta^{(n+1)} + \mathbf{u}^{(n)});$

$\mathbf{u}^{(n+1)} = \mathbf{u}^{(n)} + \beta^{(n+1)} - \mathbf{z}^{(n+1)};$

end

Algorithm 2: The ADMM optimization method [21].

183 Unlike FISTA, ADMM requires that both the proximal operator of f and
 184 Pen are computable. This is the case for the problem at hand, since f is differen-
 185 tiable, thus $\text{prox}_{\mu f}(\mathbf{v})$ is the vector $\mathbf{x}$, such that $\frac{1}{N} \mathbf{A}^T (\mathbf{A} \mathbf{x} - \mathbf{y}) + \frac{1}{\mu} (\mathbf{x} - \mathbf{v}) = \mathbf{0}$
 186 holds, that is,

$$\text{prox}_{\mu f}(\mathbf{v}) = \left(\mathbf{A}^T \mathbf{A} + \frac{N}{\mu} \mathbf{I} \right)^{-1} \left(\mathbf{A}^T \mathbf{y} + \frac{N}{\mu} \mathbf{v} \right).$$

187 Note that both methods can only be used if the closed-forms expressions
 188 of the proximal operators appearing in both algorithms are computable. Most
 189 penalty terms (Pen) have an explicit proximal operator whose expression can be
 190 derived in a straightforward manner (we recall their closed-form expressions for
 191 LASSO and Group LASSO in the supplementary material provided with this
 192 paper). Also, we find that (??) can be difficult to implement in practice, since
 193 the numerical inversion of $\mathbf{A}^T \mathbf{A} + \frac{N}{\mu} \mathbf{I}$ can be time-consuming. This is mainly
 194 due to the size of the matrix $\mathbf{A}^T \mathbf{A}$, which can be very large in the case of an
 195 overcomplete dictionary or when working on very large data sets, as typically
 196 happens in spectrographic and ECG signal processing. In Section 3.3, we shall
 197 detail a method to approximate quickly and efficiently the first step of ADMM.

198 3. FFT-Based Acceleration for Homogeneous Problems

199 From the previous discussion, it can be easily seen that [one of the main](#)
 200 [bottlenecks](#) in the implementation of Algorithm 1 and Algorithm 2 is the mul-
 201 tiplication by $\mathbf{A}$ and by $\mathbf{A}^T$, involving both $2N(N - M)p$ operations at each
 202 step. When N is large, the computations involved prevent a real-time use of
 203 such techniques. Furthermore, even if real-time processing is not required, the
 204 signal typically has to be split into several chunks in order to perform the re-
 205 construction. This can lead to artifacts in the borders of the chunk, which
 206 compromise the quality of the approximation.

207 3.1. Algorithmic Acceleration Cornerstone

As shown in [18], the steps involving multiplications by $\mathbf{A}$ and $\mathbf{A}^T$ can be substantially accelerated. Using the fact that the shapes used in $\mathbf{A}$ have a finite length, the product $\mathbf{A}_s \boldsymbol{\beta}_s$ in (??) can be seen as the terms of the discrete-time convolution between $\boldsymbol{\Gamma}_s = [\Gamma_s(t_0) \dots \Gamma_s(t_{M-1})]$ ([of size \$M\$](#)) and $\boldsymbol{\beta}_s$ ([of size \$N - M\$](#)). Thus, a fast computation can be derived using the Fast Fourier Transform (FFT) [30]. More precisely, if we denote by $\text{FFT}(\mathbf{x})$ and $\text{IFFT}(\mathbf{x})$ the N -point FFT ([with zero-padding in order to get the required size](#)) and the N -point inverse FFT of the finite-length discrete signal $\mathbf{x}$, respectively, we have:

$$\boldsymbol{\Gamma}_s * \boldsymbol{\beta}_s = \text{IFFT}(\text{FFT}(\boldsymbol{\Gamma}_s) \times \text{FFT}(\boldsymbol{\beta}_s)) ,$$

[where in the latter \$\times\$ denotes the element-wise multiplication of the elements of \$\text{FFT}\(\boldsymbol{\Gamma}_s\)\$ and \$\text{FFT}\(\boldsymbol{\beta}_s\)\$.](#) Consequently, the matrix-vector multiplication $\mathbf{A}\boldsymbol{\beta}^{(n)}$ appearing in the computation of $\nabla f(\boldsymbol{\beta})$ in (??) can be obtained with an $O(pN \log_2 N)$ computational cost, which can be a significant improvement when compared to the $O(pN(N - M))$ cost of a conventional matrix-vector multiplication. This gain can be particularly significant if $N \gg M$, which is the case in numerous signal processing applications. Similarly, the multiplication by $\mathbf{A}^T$, $\mathbf{A}^T(\mathbf{y} - \mathbf{A}\boldsymbol{\beta}^{(n)})$,

can be expressed as the concatenation of the vectors:

$$\mathbf{A}^T(\mathbf{y} - \mathbf{A}\boldsymbol{\beta}^{(n)}) = \begin{bmatrix} \mathbf{A}_1^T \mathbf{r} \\ \mathbf{A}_2^T \mathbf{r} \\ \vdots \\ \mathbf{A}_p^T \mathbf{r} \end{bmatrix},$$

where $r = \mathbf{y} - \mathbf{A}\boldsymbol{\beta}^{(n)}$. In the equation above, $\mathbf{A}_s^T \mathbf{r}$ can be interpreted as the middle terms of the discrete cross-correlation between $\boldsymbol{\Gamma}_s$ and $\mathbf{r}$ for all $s = 1 \dots p$. Therefore, each $\mathbf{A}_s^T \mathbf{r}$ can be computed by means of the FFT, with the same approach and complexity as above.

These considerations show that sparse reconstruction techniques with dictionaries based on translated waveforms can be computed in an efficient manner, since most computations can be parallelized accordingly to the number of waveforms. This makes them competitive for real-time or quasi-real-time applications. Furthermore, for off-line processing, this approach allows us to work with much larger blocks than the standard FISTA method, hence minimizing border effects and increasing the precision of the reconstruction.

3.2. FFT-based Acceleration of Proximal Gradient Methods

Besides the matrix-vector multiplications involved in proximal gradient optimization, it is well known (e.g., see [3, 21] and references therein) that the gradient step must be carefully chosen to ensure fast convergence. It was shown in [4] that the maximal step allowed in the FISTA method which can guarantee the convergence of the procedure is equal to $\text{Sp}\left(\frac{\mathbf{A}^T \mathbf{A}}{N}\right)^{-1}$, where $\text{Sp}(\cdot)$ denotes the largest eigenvalue of a given matrix. Since $\mathbf{A}^T \mathbf{A}$ is large, we cannot use in practice any decomposition technique to compute the gradient step. However, it can be approximated using power iterations. Namely, if $\mathbf{b}_0$ denotes a random vector with the same size as $\boldsymbol{\beta}$, a good approximation of an eigenvector associated with the largest eigenvalue of $\frac{\mathbf{A}^T \mathbf{A}}{N}$ can be computed using the following recursion

$$\mathbf{b}_{n+1} = \frac{\mathbf{A}^T \mathbf{A} \mathbf{b}_n}{\|\mathbf{A}^T \mathbf{A} \mathbf{b}_n\|_2}.$$

231 Note that, since $\mathbf{A}^T \mathbf{A}$ is a Gram matrix, convergence of the power iterations
 232 in (??) to an eigenvector related with $\text{Sp}\left(\frac{\mathbf{A}^T \mathbf{A}}{N}\right)$ is guaranteed. Furthermore,
 233 the computation in (??) can be performed following the same approach as be-
 234 fore, i.e., using the FFT. Consequently, the gradient's step size in FISTA can be
 235 computed rapidly and without storing the whole dictionary in memory. Simi-
 236 larly, the computation of $\nabla f(\boldsymbol{\beta}^{(n)}) = \frac{1}{N} \mathbf{A}^T (\mathbf{A} \boldsymbol{\beta}^{(n)} - \mathbf{y})$ is performed efficiently
 237 using this approach [18]. Since the rest of the FISTA algorithm involves only
 238 operations computable in linear time, we can accelerate the FISTA method sub-
 239 stantially for a variety of penalties when the dictionary is based on translated
 240 waveforms. Note, however, that the optimization procedure might diverge if
 241 the estimated value of $\text{Sp}\left(\frac{\mathbf{A}^T \mathbf{A}}{N}\right)$ is directly used for the gradient's step size.
 242 In order to compensate for the numerical error between the true and estimated
 243 value and guarantee convergence, in practical applications we should choose λ
 244 slightly lower than the estimated value of $\text{Sp}\left(\frac{\mathbf{A}^T \mathbf{A}}{N}\right)^{-1}$. As a practical rule of
 245 thumb, choosing a gradient step size equal to 0.75 times the estimated value of
 246 $\text{Sp}\left(\frac{\mathbf{A}^T \mathbf{A}}{N}\right)$ has turned out to be satisfactory for all the simulations presented
 247 in this paper.

248 3.3. ADMM Acceleration for Translated Waveforms Dictionaries

249 As discussed previously, the main bottleneck of the ADMM method is the
 250 matrix inversion in the computation of $\boldsymbol{\beta}^{n+1}$. However, as previously presented,
 251 the matrix-vector operations of the form $\mathbf{A} \mathbf{x}$ or $\mathbf{A}^T \mathbf{y}$ can be performed very
 252 efficiently due to the dictionary's specific shape. Thus, the numerical inversion
 253 of $\mathbf{A}^T \mathbf{A} + \frac{N}{\mu} \mathbf{I}$ can be performed by noting that it can be rewritten by means
 254 of a Neumann series:

$$(\mathbf{A}^T \mathbf{A} + \frac{N}{\mu} \mathbf{I})^{-1} = \frac{\mu}{N} \sum_{k=0}^{\infty} (-1)^k \left(\frac{\mu}{N} \mathbf{A}^T \mathbf{A} \right)^k.$$

255 Obviously, (??) holds provided that $\text{Sp}(\mathbf{A}^T \mathbf{A}) < N/\mu$. Thus, this decomposition
 256 cannot be used for any optimization problem. However, it can be particularly
 257 useful for all the homogeneous problems. Indeed, since in the homogeneous case
 258 the solution of $PR_{\lambda, \mathbf{A}}$ can be derived directly from the solution of $PR_{\alpha\lambda, \alpha\mathbf{A}}$,

convergence of the Neumann series can be guaranteed by scaling the matrix $\mathbf{A}^T \mathbf{A}$ by a factor of $1/\text{Sp}(\mathbf{A}^T \mathbf{A})$.

Note that the motivation for using (??) is double. First of all, it replaces the numerical inversion by a series of matrix-vector multiplications which can be performed efficiently using the aforementioned FFT-based method. Moreover, since the size of the signal to be investigated, N , is usually large in our framework, the convergence of the sum is controlled and the eigenvalues of $\frac{\mu}{N} \mathbf{A}^T \mathbf{A}$ can tend to zero rapidly. This allows us to truncate the summation in (??) after a small number of terms, depending on the dictionary's structure. It is also noteworthy that this computation can be performed only once, before the optimization procedure on the basis vectors. The results can then be cached for future uses in the optimization itself, thus yielding an additional gain in speed.

4. Numerical Results

In this section, we present results obtained on simulations for empirical validation. All the simulations were performed on a desktop PC with an i7 processor and 32 GB of RAM.

4.1. Simulations Overview

In the presented simulations we focus on two main practical aspects: the execution time and the quality of the sparse regressor obtained. Regarding the first matter, we have investigated the performance of both the proximal gradient method and ADMM on two standard sparse approximation procedures: LASSO [19] and Group LASSO [20]. We performed 100 experiments in each case in order to be able to show both the average and the standard deviation of the results. For each experiment, we simulated a sparse signal based on ten translated, randomly chosen, shapes made of Gaussian noise with zero mean and standard deviations ranging from 1 to 5, and with random amplitudes chosen from a uniform distribution on $[30, 40]$. The locations of the shapes correspond to the arrival times of an homogeneous Poisson process with intensity 0.02. An

287 additional Gaussian noise with standard deviation of $\sigma = 0.1$ was added to
 288 the resulting synthetic signal. An example of an obtained signal of size 100 is
 289 displayed in Figure 1.

290 For all the experiments, we recorded the elapsed time and the relative error
 291 between fast implementation and regular implementation of each method. These
 292 experiments were repeated for different noise variances, as well as for different
 293 sizes of the signals N ranging from 200 to 80000. For large values of N , the
 294 main bottleneck of ADMM (namely, the numerical inversion of $\mathbf{A}^T \mathbf{A} + \frac{N}{\lambda} \mathbf{I}$)
 295 prevents the computation of the regular implementation due to lack of memory.
 296 Similarly, for very large values of N , the full storage of $\mathbf{A}$ becomes unfeasible,
 297 implying that the regular proximal gradient cannot be computed. In both cases,
 298 results for the regular implementation are omitted.

299 We also investigated the influence of the approximation made by replac-
 300 ing (??) with (??) in the fast implementation of ADMM. Obviously, the sum-
 301 mation in (??) must be truncated in practical implementations. Therefore, the
 302 truncation index must be carefully chosen in order to provide a good compro-
 303 mise between speed and quality of the estimation. In order to investigate this
 304 loss of precision, we measured (when possible) the relative error between the
 305 regressor obtained by standard proximal optimization (whether the method be
 306 the proximal gradient optimization or ADMM), and the regressor obtained by
 307 means of the proposed acceleration, that is

$$\frac{\|\hat{\beta}_{\text{fast}} - \hat{\beta}_{\text{normal}}\|_2}{\|\hat{\beta}_{\text{normal}}\|_2}$$

308 4.2. Simulation Results and Discussion

309 Results on execution times are displayed for the LASSO penalty in Figure 2
 310 using the proximal gradient, and in Figure 3 using ADMM. Results on the exe-
 311 cution times are displayed for the Group LASSO in Figure 4 using the proximal
 312 gradient, and in Figure 5 using ADMM. The precision losses for the different
 313 settings are displayed in Figure 6 for N varying from 200 to 1100.

314 From the results obtained in Figures 2, 3, 4 and 5, we note an improvement
 315 in execution times in all cases. This improvement is more significant for ADMM

316 (e.g., it is more than 200 times faster for $N = 2000$) than for the proximal gradi-
 317 ent, although in this case the algorithm is also faster in its new version. This is to
 318 be expected, considering that tests of running times have been performed for rel-
 319 atively small lengths in order to allow for the comparison between the standard
 320 and the new implementation. In these cases the gain obtained by replacing $\mathbf{A}\beta$
 321 and $\mathbf{A}^T \mathbf{r}$ with FFT-based operations is small. In contrast, the ADMM includes
 322 the inversion of one Gram matrix, which has an $O(N^3 p^3)$ complexity. Even if
 323 this inversion is performed only once, this step of ADMM becomes extremely
 324 cumbersome as the size of the signal increases. In this case, the gain in speed
 325 corresponds to replacing an operation of complexity $O(N^3 p^3)$ by one operation
 326 of complexity $O(KNp \log_2(Np))$, which is significant. Furthermore, note that
 327 our fast implementation allows us to process blocks of $N = 80000$ samples (the
 328 maximum signal length considered in this work), whereas the standard imple-
 329 mentation becomes unfeasible for $N > 8000$ and $N > 2000$, respectively, for the
 330 proximal gradient and ADMM approaches.

331 The error between the two proximal gradient algorithms is equal to the
 332 machine's precision. This fact can be easily explained, since in this case we
 333 use the FFT instead of regular vector matrix multiplications, which are not an
 334 approximation per se. Therefore, we can say that the use of the improvement of
 335 the proximal gradient has a negligible effect on the quality of the obtained sparse
 336 regressor. In contrast, since the ADMM upgrade involves a series truncation
 337 in order to replace the matrix inversion, we attain a larger error. However, in
 338 Figure 6 we see that this error decreases as the length of the signal increases.
 339 This is understood by the fact that the larger the size of the $\mathbf{A}^T \mathbf{A}$ matrix, the
 340 smaller λ/N is. Therefore, the convergence of the series approximation in (??)
 341 becomes faster and faster.

342 In Figure 7, we investigate the influence of the gradient step μ for proximal
 343 gradient methods, based on (??). We have observed, from our experiments,
 344 that a good automatic choice of the gradient step can indeed improve the speed
 345 of the algorithm. However, since it is a refinement in the method itself, and not
 346 a conceptual change, the algorithmic complexity remains the same. Therefore,

the slope of the obtain graphs is the same in both cases. On a similar note, recent publications [31, 32] show that ADMM can be further accelerated if a larger step-length is used in the update of $\mathbf{u}^{(n)}$. More specifically, [32] shows that this step length can be increased up to $\frac{1+\sqrt{5}}{2}$. This idea should be investigated further in future contributions.

5. Conclusion

In this paper, we have presented a fast implementation of two well-known and widely used optimization schemes in the field of compressive sensing. Numerical experiments have shown that both proximal gradient methods and ADMM can be made significantly faster when dealing with dictionaries composed of translated waveforms, and the proposed implementation allows us to reconstruct very large signals when standard implementations fail. Future contributions will detail the use of these implementations for the analysis of biomedical data and spectroscopic signals.

Acknowledgements

We would like to thank the reviewers for their careful reading and remarks, which greatly improved the quality of this contribution. Part of this research was funded by the ERASMUS+ BIOART program.

References

- [1] I. Daubechies, M. Defrise, C. D. Mol, An iterative thresholding algorithm for linear inverse problems with a sparsity constraint, Communications in Pure and Applied Mathematics 57 (2004) 1413–1457.
- [2] I. E. Nesterov, A Method for Solving the Convex Programming Problem with Convergence Rate $O(1/k^2)$, Dokl. Akad. Nauk SSSR (in Russian) 269 (1983) 543–547.

- [3] A. Beck, M. Teboulle, A fast iterative shrinkage-thresholding algorithm for linear inverse problems, *SIAM Journal on Imaging Sciences* 2 (2009) 183–202.
- [4] A. Beck, M. Teboulle, Gradient-based algorithms with applications to signal recovery, in: D. P. Palomar, Y. C. Eldar (Eds.), *Convex optimization in signal processing and communications*, Cambridge university press, Cambridge, UK, 2010, Ch. 2, pp. 42–88.
- [5] Y. Sepulcre, T. Trigano, Y. Ritov, Sparse regression algorithm for activity estimation in gamma spectrometry, *IEEE Transactions on Signal Processing* 61 (17) (2013) 4347–4359. doi:10.1109/TSP.2013.2264811.
- [6] T. Trigano, Y. Sepulcre, Y. Ritov, Sparse Reconstruction Algorithm for Nonhomogeneous Counting Rate Estimation, *IEEE Transactions on Signal Processing* 65 (2) (2017) 372–385.
- [7] T. Trigano, J. Cohen, Intensity Estimation of Spectroscopic Signals With an Improved Sparse Reconstruction Algorithm, *IEEE Signal Processing Letters* 24 (5) (2017) 530–534. doi:10.1109/LSP.2017.2680839.
- [8] X. Huang, Y. Liu, L. Shi, S. Huffel, J. Suykens, Two-Level ℓ_1 Minimization for Compressed Sensing , *Signal Processing* 108 (2015) 459 – 475.
- [9] L. Nie, D. Jiang, Z. Xu, A compressive sensing-based reconstruction approach to network traffic, *Computers & Electrical Engineering* 39 (5) (2013) 1422–1432.
- [10] M. Sorel, F. Sroubek, Fast convolutional sparse coding using matrix inversion lemma, *Digital Signal Processing* 55 (2016) 44–51.
- [11] A. Ghaffari, H. Palangi, M. Babaie-Zadeh, C. Jutten, ECG denoising and compression by sparse 2D separable transform with overcomplete mixed dictionaries, in: *Proc. 2009 IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, 2009, pp. 1–6.

- 399 [12] I. Ciocoiu, Single channel fetal ECG recovery using sparse redundant rep-
400 resentations, in: Proc. 10th International Symposium on Signals, Circuits
401 and Systems (ISSCS), 2011, pp. 1–4.
- 402 [13] S. Monzón, T. Trigano, D. Luengo, A. Artés-Rodríguez, Sparse spectral
403 analysis of atrial fibrillation electrograms, in: Proc. 2012 IEEE Interna-
404 tional Workshop on Machine Learning for Signal Processing (MLSP), 2012,
405 pp. 1–6.
- 406 [14] D. Luengo, J. Vía, S. Monzón, T. Trigano, A. Artés-Rodríguez, Cross-
407 products LASSO, in: Proc. 2013 IEEE International Conference on Acous-
408 tics, Speech and Signal Processing (ICASSP), 2013, pp. 6118–6122.
- 409 [15] Z. Zhang, T.-P. Jung, S. Makeig, B. Rao, Compressed sensing for energy-
410 efficient wireless telemonitoring of noninvasive fetal ECG via block sparse
411 Bayesian learning, IEEE Transactions on Biomedical Engineering 60 (2)
412 (2013) 300–309. doi:10.1109/TBME.2012.2226175.
- 413 [16] D. Luengo, S. Monzón, T. Trigano, J. Vía, A. Artés-Rodríguez, Blind anal-
414 ysis of atrial fibrillation electrograms: a sparsity-aware formulation, Inte-
415 grated Computer-Aided Engineering 22 (1) (2015) 71–85.
- 416 [17] P. Buhlmann, P. Rutimann, S. van de Geer, C.-H. Zhang, Correlated vari-
417 ables in regression: Clustering and sparse estimation, Journal of Statistical
418 Planning and Inference 143 (11) (2012) 1835–1858.
- 419 [18] T. Trigano, I. Shevtsov, D. Luengo, Cosa: An Accelerated ISTA Algorithm
420 for Dictionaries Based on Translated Waveforms, Signal Processing 139
421 (2017) 131–135.
- 422 [19] R. Tibshirani, Regression shrinkage and selection via the LASSO, Journal
423 of the Royal Statistical Society 58 (1) (1996) 267–288.
- 424 [20] M. Yuan, Y. Lin, Model selection and estimation in regression with grouped
425 variables, Journal of the Royal Statistical Society, B. 68 (1) (2006) 49–67.

- 426 [21] N. Parikh, S. Boyd, Proximal Algorithms, *Foundations and Trends in Op-*
427 *timization* 1 (3) (2014) 127–239.
- 428 [22] S. Mosci, L. Rosasco, M. Santoro, A. Verri, S. Villa, Solving structured
429 sparsity regularization with proximal methods, in: *Proceedings of the*
430 *2010 European conference on Machine learning and knowledge discovery*
431 *in databases: Part II, ECML PKDD’10*, Springer-Verlag, Berlin, Heidel-
432 berg, 2010, pp. 418–433.
- 433 [23] V. Pappyan, Y. Romano, J. Sulam, M. Elad, Theoretical Foundations of
434 Deep Learning via Sparse Representation, *Signal Processing Magazine* 1 (7)
435 (2018) 72–89.
- 436 [24] M. Jas, T. Dupré La Tour, U. Simsekli, A. Gramfort, Learning the Mor-
437 phology of Brain Signals Using Alpha-Stable Convolutional Sparse Coding,
438 in: *Advances in neural information processing systems*, Long Beach, United
439 States, 2017.
- 440 [25] Y. Zhu, S. Lucey, Convolutional Sparse Coding for Trajectory Reconstruc-
441 tion, *IEEE Transactions on Pattern Analysis and Machine Intelligence*
442 37 (3) (2015) 529–540.
- 443 [26] P. Jao, L. Su, Y. Yang, B. Wohlberg, Monaural Music Source Separation
444 Using Convolutional Sparse Coding, *IEEE/ACM Transactions on Audio,*
445 *Speech, and Language Processing* 24 (11) (2016) 2158–2170.
- 446 [27] A. Cogliati, Z. Duan, B. Wohlberg, Context-Dependent Piano Music Tran-
447 scription With Convolutional Sparse Coding, *IEEE/ACM Transactions on*
448 *Audio, Speech, and Language Processing* 24 (12) (2016) 2218–2230.
- 449 [28] Y. Wang, Q. Yao, J. T. Kwok, L. M. Ni, Scalable Online Convolutional
450 Sparse Coding, *IEEE Transactions on Image Processing* 27 (10) (2018)
451 4850–4859.

- [29] B. Wohlberg, Efficient convolutional sparse coding, in: 2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2014, pp. 7173–7177.
- [30] A. V. Oppenheim, R. W. Schaffer, J. R. Buck, Discrete-time Signal Processing (2nd Ed.), Prentice-Hall, Upper Saddle River, NJ, USA, 1999.
- [31] Z. Wen, D. Goldfarb, W. Yin, Alternating Direction Augmented Lagrangian Methods for Semidefinite Programming, Mathematical Programming Computation 2 (3) (2010) 203–230.
- [32] J. Xie, On Inexact ADMMs with Relative Error Criteria, Computational Optimization and Applications 71 (3) (2018) 743–765.
- [33] S. Mosci, S. Villa, A. Verri, L. Rosasco, A primal-dual algorithm for group sparse regularization with overlapping groups, in: Advances in Neural Information Processing Systems 23, 2010, pp. 2604–2612.

6. Supplementary Material

We recall here a few properties of proximal operators that are used in this paper. Most of this section’s proofs can be found in [21].

6.1. Additional Results on Proximal Operators

Roughly speaking, (??) can be understood either as the generalization of a projection operator or as a compromise between minimizing g while remaining close to $\mathbf{v}$. The cornerstone of proximal algorithms for optimization is that $\mathbf{x}^*$ is a minimizer of g if and only if it is a fixed point of $\mathbf{prox}_g$, that is $\mathbf{prox}_g(\mathbf{x}^*) = \mathbf{x}^*$. Therefore, a possible, preliminary algorithm for minimizing g would be to iterate the proximal operator until convergence:

$$\mathbf{x}^{(n+1)} = \mathbf{prox}_{\mu g}(\mathbf{x}^{(n)}),$$

where μ denotes a step size controlling how close to $\mathbf{x}^{(n)}$ we wish to remain at each iteration. Though $\mathbf{prox}_{\mu f}$ is not a contracting, but only a non-expansive,

operator, it turns out that (??) can be adapted to a converging algorithm. We refer to [21, Chapter 3] for an in-depth explanation on this issue and how to solve it. For further explanation, we can also define the subdifferential set of any function g as

$$\partial g(\mathbf{x}) = \left\{ \mathbf{y} ; g(\mathbf{z}) \geq g(\mathbf{x}) + \mathbf{y}^T(\mathbf{z} - \mathbf{x}), \mathbf{z} \in D_g \right\}.$$

If g is differentiable on $\mathbf{x}$, then $\partial g(\mathbf{x})$ in (??) reduces to the singleton $\{\nabla g(\mathbf{x})\}$, and that $\mathbf{x}$ is the minimizer of g if and only if $0 \in \partial g(\mathbf{x})$. Thus, the subdifferential set can be understood as an extension of the differential operator. In addition, it is useful to find the minimizer of convex functions which are non necessarily differentiable, as it happens in our case.

Proximal optimization methods applied on sparse regression problems rely on two important properties of the proximal operator:

1. $\mathbf{prox}_{\mu g} = (\mathbf{I} + \mu \partial g)^{-1}$
2. the separability property, namely if g is fully separable ($g(\mathbf{x}) = \sum_{i=1}^n g_i(x_i)$), then $(\mathbf{prox}_g(\mathbf{v}))_i = (\mathbf{prox}_{g_i}(v_i))$.

As detailed in [21], $(\mathbf{I} + \mu \partial g)^{-1}$ is a relation which is single valued, and therefore can be equated with a function even if ∂g is not. The second property allows us to get a closed-form term for the proximal operator of Pen in many cases, for example for LASSO, Grouped LASSO and other related penalties. This stems from the form of the penalties introduced, which rely on ℓ_1 or ℓ_2 norms. We now describe two very commonly used algorithms to solve (??) in practice: the proximal gradient method and the ADMM.

Let us denote by $\hat{\beta}$ a minimizer of (??), whose existence is guaranteed by convexity. Then we have for all $\mu > 0$:

$$\begin{aligned} \mathbf{0} \in \partial(\mu f + \lambda \mu \text{Pen})(\hat{\beta}) &\Leftrightarrow \mathbf{0} \in \mu \nabla f(\hat{\beta}) + \lambda \mu \partial \text{Pen}(\hat{\beta}) \\ &\Leftrightarrow \hat{\beta} - \mu \nabla f(\hat{\beta}) \in (\mathbf{I} + \lambda \mu \partial \text{Pen})(\hat{\beta}). \end{aligned}$$

Using the first property with (??), we can observe that $\hat{\beta} = \mathbf{prox}_{\mu \lambda \text{Pen}}(\hat{\beta} - \mu \nabla f(\hat{\beta}))$. Thus, a proximal gradient algorithm to obtain the solution of (??) is (??).

503 As stated in the paper, the usefulness of the presented approach depends
504 strongly on whether $\mathbf{prox}_{\lambda\mu\text{Pen}}$ is easily computable or not. Fortunately, this is
505 the case for a wide variety of sparsity-driven regression problems. Due to the
506 separability property, it can be shown that, for the LASSO penalty ($\text{Pen}(\boldsymbol{\beta}) =$
507 $\|\boldsymbol{\beta}\|_1$) we have

$$\mathbf{prox}_{\lambda\mu\text{Pen}}(\mathbf{v}) = \begin{cases} v_i - \lambda\mu, & \text{if } v_i > \lambda\mu; \\ v_i + \lambda\mu, & \text{if } v_i < -\lambda\mu; \\ 0, & \text{if } |v_i| \leq \lambda\mu; \end{cases}$$

508 whereas the explicit computation for the grouped LASSO penalty ($\text{Pen}(\boldsymbol{\beta}) =$
509 $\sum_{g \in \mathcal{G}} \|\boldsymbol{\beta}_g\|_2$) yields

$$(\mathbf{prox}_{\lambda\mu\text{Pen}}(\mathbf{v}))_g = \max \left\{ 0, 1 - \frac{\lambda\mu}{\|\mathbf{v}_g\|_2} \right\} \mathbf{v}_g.$$

510 The explicit computation of (??) was performed in [21, 22], while [33] derived
511 the proximal operator for the penalty of the grouped LASSO with overlapping
512 groups. Note that, even in this case (see [33] and references therein), the latter
513 can be addressed using (??) with a dictionary including duplications of the
514 dictionary's columns appearing more than once.

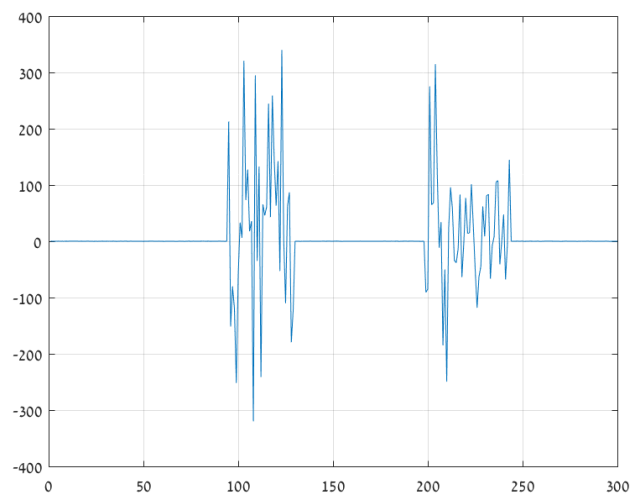


Figure 1: Example of generated synthetic signal

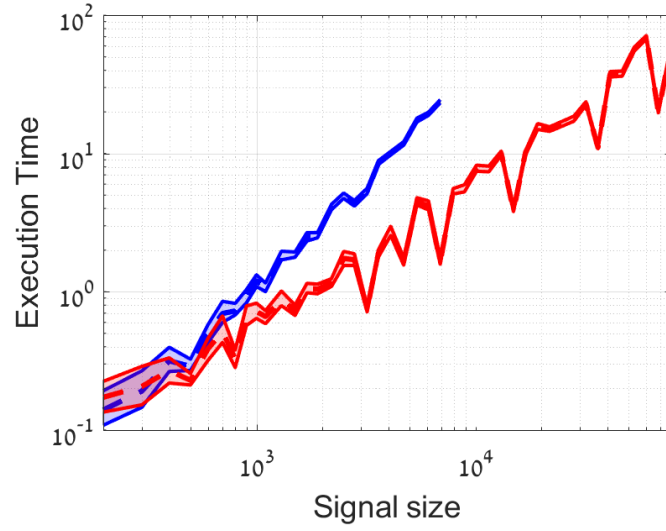


Figure 2: Average execution times with 90% confidence interval for LASSO obtained by proximal gradient optimization, for different dictionary sizes. Blue: standard implementation. Red: Fast implementation.

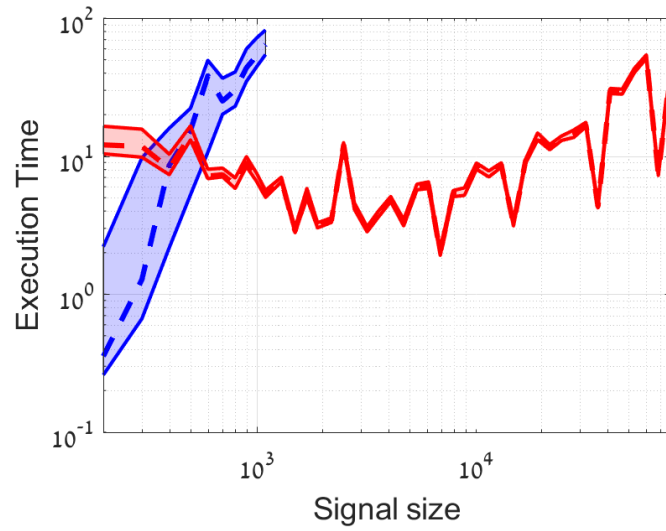


Figure 3: Average execution times with 90% confidence interval for LASSO obtained by ADMM, for different dictionary sizes. Blue: standard implementation. Red: Fast implementation.

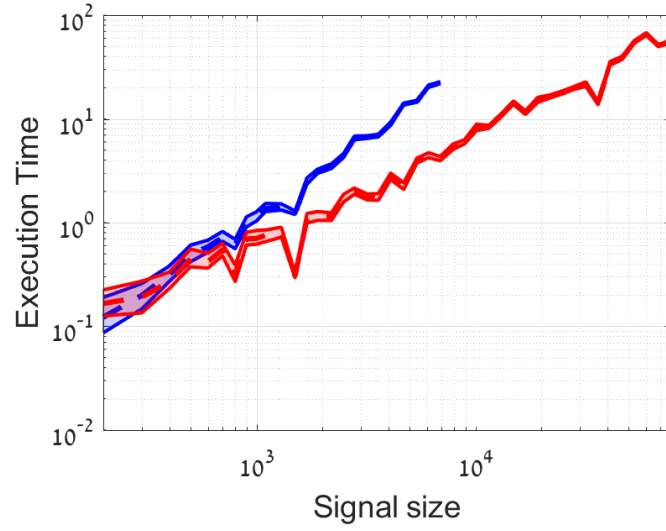


Figure 4: Average execution times with 90% confidence interval for Grouped LASSO obtained by proximal gradient optimization, for different dictionary sizes. Blue: standard implementation. Red: Fast implementation.

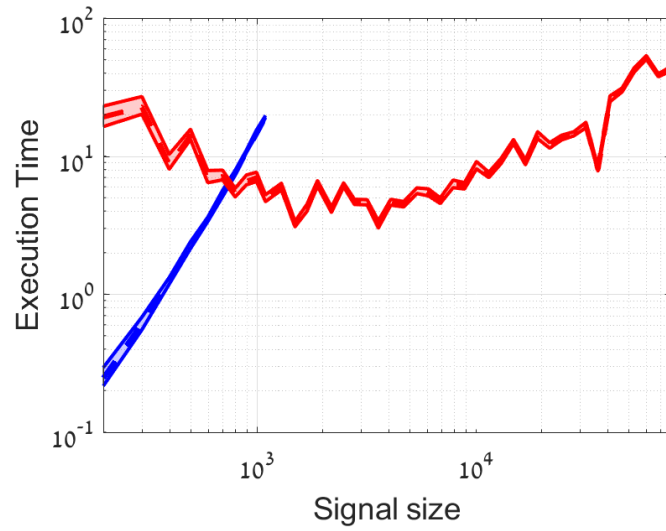


Figure 5: Average execution times with 90% confidence interval for Grouped LASSO obtained by ADMM, for different dictionary sizes. Blue: standard implementation. Red: Fast implementation.

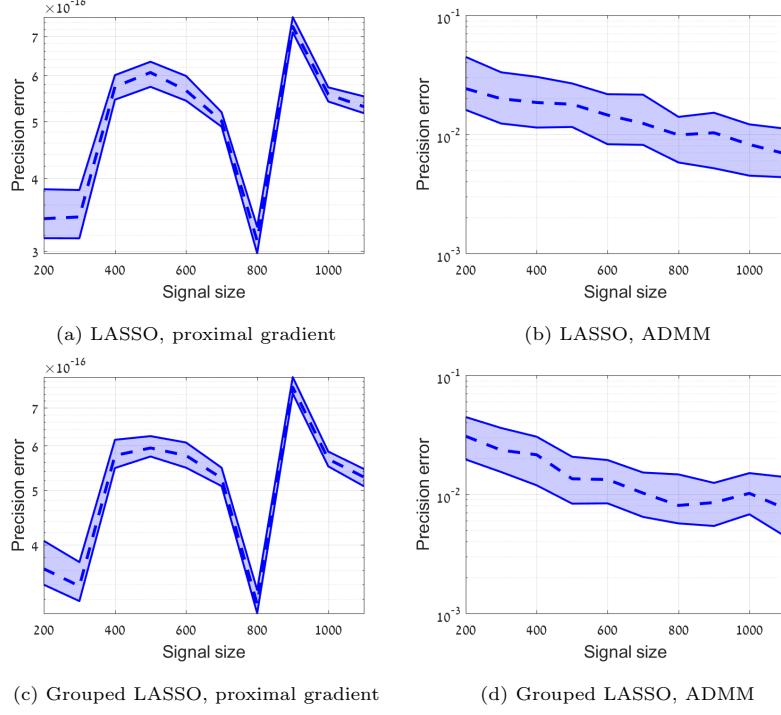


Figure 6: Precision losses between the regular optimization and the fast optimization

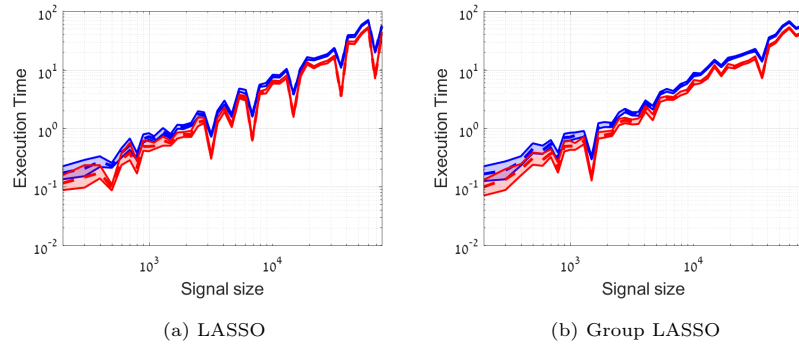


Figure 7: Influence of the gradient step on proximal gradient based algorithm. Blue: manual setting, set up to $\mu = 0.0003$. Red: automatic setting, computed based on (??).